

# Comprehensive System Verification & Debugging Report

Advanced Corner-Case Testing & Bug Resolution

Ziad Mostafa Abdelaziz

September 15, 2026

## Contents

|          |                                                            |          |
|----------|------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                        | <b>2</b> |
| <b>2</b> | <b>Testbench Evolution: Jitter &amp; Time Generation</b>   | <b>2</b> |
| 2.1      | The Challenge: Realistic CDC Stressing . . . . .           | 2        |
| 2.2      | Error 1: ModelSim Scheduling with Real Variables . . . . . | 2        |
| <b>3</b> | <b>Testbench Synchronization: The Stale Data Catch</b>     | <b>2</b> |
| 3.1      | Error 2: Hanging Waits and Simulation Pollution . . . . .  | 2        |
| <b>4</b> | <b>RTL Design Bug 1: The UART TX CDC Deadlock</b>          | <b>3</b> |
| 4.1      | The Symptom . . . . .                                      | 3        |
| 4.2      | The Deep Dive . . . . .                                    | 3        |
| 4.3      | The Root Cause . . . . .                                   | 3        |
| 4.4      | The Fix . . . . .                                          | 4        |
| <b>5</b> | <b>RTL Design Bug 2: SYS_CTRL ALU Clock Gating Defect</b>  | <b>4</b> |
| 5.1      | The Symptom . . . . .                                      | 4        |
| 5.2      | The Deep Dive . . . . .                                    | 4        |
| 5.3      | The Root Cause . . . . .                                   | 5        |
| 5.4      | The Fix . . . . .                                          | 5        |
| <b>6</b> | <b>Conclusion</b>                                          | <b>5</b> |

# 1 Introduction

This document serves as a comprehensive log of the trials, errors, debugging processes, and final resolutions encountered during the advanced verification of the Top-Level System (SYS\_TOP). Our primary goal was to subject the system to an advanced corner-case testbench designed to stress Clock Domain Crossing (CDC), handle asynchronous resets, test parity framing, and verify synchronization between clock domains.

During the execution of this robust test suite, a series of complex failures emerged. Some failures were traced back to the testbench's interaction with the simulator's scheduling mechanisms, while others revealed critical, deeply hidden flaws within the System's RTL design.

This report details the thought processes used to track down each issue, the root-cause analysis (RCA), and the precise corrective measures applied.

## 2 Testbench Evolution: Jitter & Time Generation

### 2.1 The Challenge: Realistic CDC Stressing

To truly verify a system bridging two disparate clock domains (REF\_CLK at 50 MHz and UART\_CLK at 3.6864 MHz), the testbench needed to emulate realistic clock jitter and phase drift. The initial attempt utilized randomized `#delay` constructs and `real` data types.

### 2.2 Error 1: ModelSim Scheduling with Real Variables

**The Symptom:** The testbench implemented a `current_bit_period` variable of type `real` to calculate the baud rate on the fly based on injected jitter. However, when the UART framing task (`send_uart_frame`) fired, the bit periods calculated were completely zeroed out (0.0), causing simultaneous transitions on the RX\_IN line.

**The Investigation:** We traced the value of `current_bit_period` via simulation logs. We noticed that although the variable was assigned within a combinatorial block (`always @(*)`), it was failing to trigger a re-evaluation when its dependencies changed. In Verilog, continuous assignments or combinatorial blocks involving `real` types are notoriously problematic in legacy simulators like ModelSim ASE, which often fail to schedule evaluation events properly for reals.

**The Fix:** Instead of relying on an `always @(*)` block, we converted the timing logic into procedural assignments. More importantly, we introduced a highly deterministic custom task: `send_uart_frame_custom`. This task explicitly consumed delays based on the exact `current_bit_period` parameter passed dynamically. This guaranteed that the RX\_IN stimulus correctly reflected the sub-nanosecond jitter injected.

## 3 Testbench Synchronization: The Stale Data Catch

### 3.1 Error 2: Hanging Waits and Simulation Pollution

**The Symptom:** As the testbench progressed to Test 5 and beyond, it repeatedly suffered from catastrophic timeouts (e.g., reaching the 43402720000 ps global timeout). Furthermore, the ModelSim transcript was hopelessly flooded with `$display` statements printing every clock cycle.

**The Investigation:** First, we had to address the transcript pollution. The original author embedded unconditional `$display` statements in purely combinatorial `always @(*)` blocks within `SYS_CTRL.v` and `RegFile.v`. Because these blocks executed on every minor input transition, the simulator was bottlenecked by file I/O operations, making trace analysis nearly impossible.

After creating a pristine `fresh_stress_run` directory and stripping out the `$display` pollution, the root synchronization bug became evident. The testbench task `receive_byte` was blindly executing `wait (!TX_OUT)`. If the `ASYNC_FIFO` hadn't fully drained from a previous back-to-back command (e.g., Test 4), `TX_OUT` would transmit a stale byte, which the testbench would erroneously catch, misaligning the entire test sequence. Worse, if a stop-bit framing error occurred (Test 3) and no byte was transmitted, the `wait` statement hung indefinitely.

**The Fix:** 1. **Sanitized RTL:** Removed combinatorial prints from the RTL, relying solely on waveform dumps and structural `test_bench` logging. 2. **Timeout Protection:** Upgraded the `receive_byte` task with explicit wait loop limits. 3. **State Flushing:** Implemented mandatory idle-wait intervals (`#(BIT_PERIOD * 15)`) between tests and synchronized against `fifo_empty == 1` to guarantee that each stress test began with a completely clean slate.

## 4 RTL Design Bug 1: The UART TX CDC Deadlock

### 4.1 The Symptom

Even with a perfectly synchronized testbench, Test 5 completely failed. The testbench reported receiving '100' (which is  $0 \times 64$  in hexadecimal) **twice**. The `UART_TX` module was transmitting the byte  $0 \times 64$  over and over in an infinite loop, starving the system of the second byte ( $0 \times 00$ ) and trapping the state machine in a deadlock.

### 4.2 The Deep Dive

We extracted the FSM trace for the `UART_TX` block. The protocol flow for reading data from the `ASYNC_FIFO` was as follows: 1. `UART_TX` FSM resides in `IDLE`. 2. `ASYNC_FIFO` asserts `Data_Valid` (i.e., `~fifo_empty`). 3. FSM asserts `Busy` and transitions to `START`, `DATA`, `PAR`, and `STOP`. 4. On the transition to `STOP`, the system generates a pulse (`fifo_r_inc`) to pop the FIFO.

We inspected `PULSE_GEN.v`. It generates the `fifo_r_inc` pulse exclusively on the **falling edge** of the `Busy` signal:

```
always @(posedge CLK) q_reg <= LVL_SIG; // LVL_SIG is Busy
assign PULSE_SIG = q_reg & ~LVL_SIG;
```

We then inspected the FSM in `uart_fsm.v`:

```
STOP: begin
    ser_en  = 1'b0;
    mux_sel = 2'b01;
    Busy    = 1'b1; // Busy remains HIGH!
    if (Data_Valid)
        next_state = START;
    else
        next_state = IDLE;
end
```

### 4.3 The Root Cause

This was a classic deadlock! In the `STOP` state, `Busy` remained asserted (`1'b1`). If the FIFO held multiple bytes, `Data_Valid` was still true, meaning the FSM immediately looped back to `START`. Because `Busy` never went low, `PULSE_GEN` never detected a falling edge, and thus `fifo_r_inc`

was never asserted. The FIFO was never popped, causing the FSM to infinitely re-transmit the exact same byte ( $0 \times 64$ ).

## 4.4 The Fix

We modified `uart_fsm.v` to force a single-cycle transition to IDLE unconditionally, allowing Busy to drop:

```
STOP: begin
    ser_en  = 1'b0;
    mux_sel = 2'b01;
    Busy    = 1'b0; // Drop Busy
    next_state = IDLE; // Unconditional transition
end
```

This single cycle of IDLE flawlessly pulsed `fifo_r_inc`, popped the FIFO, loaded the new byte into the serializer, and broke the infinite loop.

# 5 RTL Design Bug 2: SYS\_CTRL ALU Clock Gating Defect

## 5.1 The Symptom

With Test 5 passing, we moved to Test 6: an ALU Subtraction of  $5 - 16$ . The expected result was  $-11$  ( $0xFFFF5$ ). Incredibly, the testbench received  $0x0064$  (100 in decimal)—the exact result of Test 5's addition!

## 5.2 The Deep Dive

We verified via trace logs that `SYS_CTRL` received the bytes perfectly. The operands were correctly parsed: `OP_A = 5`, `OP_B = 16`, `ALU_FUN = 1`. The `SYS_CTRL` state machine entered `ALU_EXEC`, processed it, and transitioned to `TX_BYTE1`.

If the inputs were correct, why was the output stale? We looked closely at `ALU.v` and its interaction with clock gating:

```
// In ALU.v
always @(posedge CLK or negedge RST) begin
    if (Enable) begin
        ALU_OUT  <= alu_comb_result;
        OUT_VALID <= 1'b1;
    end else begin
        OUT_VALID <= 1'b0; // Clears validation flag
    end
end
```

The clock driving the ALU, `GATED_CLK`, was provided by `SYS_CTRL.v`:

```
// In SYS_CTRL.v
ALU_EXEC: begin
    CLK_EN = 1'b1;
    EN     = 1'b1;
    ALU_FUN = alu_fun_reg;
    if (OUT_Valid)
        next_state = TX_BYTE1;
end
```

### 5.3 The Root Cause

This was an insidious clock-gating bug. When `ALU.v` finished the computation, `OUT_VALID` went high. On the immediate next clock cycle, `SYS_CTRL` recognized the flag and transitioned to `TX_BYTE1`. Because the state changed, `SYS_CTRL` immediately de-asserted `CLK_EN`, dropping it to 0. This instantly stopped the `GATED_CLK`.

Crucially, because the clock stopped, `ALU.v` never received a clock edge where `Enable` was 0. As a result, its `OUT_VALID` register was permanently stuck at 1! When Test 6 initiated a subtraction, `SYS_CTRL` entered `ALU_EXEC` and saw that `OUT_VALID` was **already high**. It immediately aborted the wait and captured the old `ALU_OUT` (0x0064) before the ALU had the chance to process the new subtraction.

### 5.4 The Fix

We needed to grant the ALU one extra clock cycle with `Enable = 0` to clear its registers. We achieved this by asserting `CLK_EN` during the `TX_BYTE1` state:

```
TX_BYTE1: begin
    TX_D_VLD = 1'b1;
    CLK_EN   = 1'b1; // Extra tick to clear OUT_VALID
    // ...
end
```

This elegant fix gave the ALU exactly one tick to see `EN = 0` and successfully clear `OUT_VALID`.

## 6 Conclusion

Following these critical structural fixes, the advanced stress test suite was executed in full. All 7 test cases—ranging from start-bit glitches to severe phase drift, back-to-back command saturation, parity corruption, and asynchronous resets—passed with 100% success.

The integration of advanced CDC verification techniques systematically exposed hidden RTL flaws. By relying on rigorous simulation logs, edge-tracing, and logic-analyzer methodologies, we hardened the ASIC Flow design against catastrophic field failures.